

Persistencia de Datos almacenamiento y respaldo

Sesión 4 · Que los datos sobrevivan a cualquier contenedor

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida

Docente: Jawy Andrés Romero Pinto

MSc · Especialista en ciberseguridad y nube híbrida · LFCE

La pregunta que quedó sembrada

Y el experimento cruel con el que abrimos hoy

Lo que pasó en la S3

- Mataste el contenedor del blog... y el post sobrevivió.
- Porque los datos no viven en el blog: viven en la db.
- **Pero la db también es un contenedor. Y todo contenedor es EFÍMERO (S1).**

Hoy (S4)

- Primero lo comprobamos: matamos la db CON los posts adentro. 💀
- Luego lo arreglamos: volúmenes — datos fuera del ciclo de vida del contenedor.
- Y lo blindamos: respaldo y restauración PROBADA.
- 🚀 **Además: arranca el mini-proyecto integrador.**

Entregable de hoy: tu blog con volumen persistente + un respaldo restaurado con éxito (probado, no prometido).

Objetivos de la sesión

Al final de estas 2 horas podrás...

Explicar

por qué los contenedores pierden sus datos: la capa de escritura efímera.

Persistir

los datos en volúmenes que sobreviven a la muerte de cualquier contenedor.

Respaldar

y restaurar la información de tus contenedores — con prueba de restauración.

Diseñar

una estrategia mínima de respaldo (3-2-1) para una app contenerizada.

Por qué se pierden los datos

La capa de escritura: el secreto que ya conoces desde la S2

CAPA DE ESCRITURA · lo que la app escribe (¡EFÍMERA!)

capas de la IMAGEN · solo lectura, inmutables

base alpine · solo lectura

*docker rm = la capa de escritura se destruye.
La imagen queda intacta; TUS DATOS no.*

Ya lo viviste dos veces

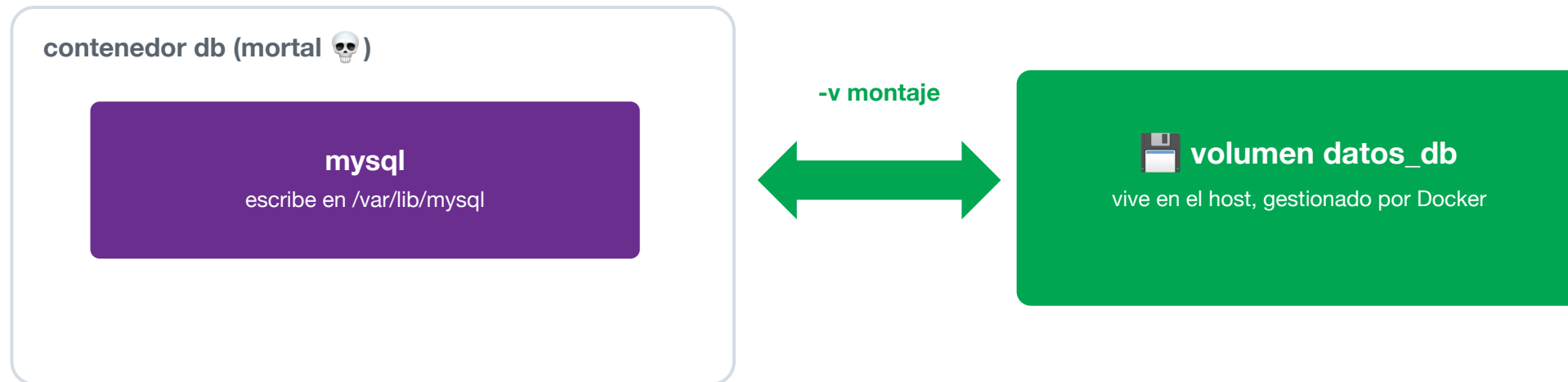
- S1: /huella.txt desapareció con el contenedor.
- S3: el post sobrevivió al blog... porque estaba en OTRO contenedor (la db).
- Hoy: si muere la db, mueren los posts — la capa de escritura de mysql se va con ella.

La imagen es la receta; la capa de escritura es un borrador desechable. Los datos importantes necesitan OTRA casa.

Volúmenes: la casa de los datos

Almacenamiento gestionado por Docker, FUERA del ciclo de vida del contenedor

```
docker volume create datos_db  
docker run -d --name db -v datos_db:/var/lib/mysql mysql:8.0
```



El contenedor muere; el volumen NO. El siguiente contenedor que lo monte encuentra los datos exactamente donde quedaron.

Volumen o carpeta del host (bind mount)

Dos formas de montar — cuándo usar cada una

Volumen (-v nombre:/ruta)

- Docker lo crea y lo administra (docker volume ls).
- Portable entre hosts: no depende de rutas locales.
- Ideal para DATOS de la aplicación: bases de datos, archivos subidos.
- **El estándar en producción. El que usamos hoy.**

Bind mount (-v /ruta/host:/ruta)

- Monta una carpeta EXACTA del host dentro del contenedor.
- Perfecto en desarrollo: editas el código afuera, el contenedor lo ve al instante.
- Frágil entre máquinas (la ruta debe existir) y  riesgoso si montas rutas sensibles del host.
- **Lo usarás hoy para SACAR el respaldo del contenedor.**

Regla simple: datos de la app → volumen · archivos que tú editas o extraes → bind mount.

El blog inmortal: la arquitectura de hoy

La S3 + una línea: el volumen de la base de datos

```
docker network create ean-net
docker volume create datos_db
docker run -d --name db --network ean-net -v datos_db:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=... mysql:8.0
```



Hoy lo pruebas: creas un post, ejecutas `docker rm -f db`, recreas la db... y el post sigue ahí. La app completa se vuelve desechable; los datos, no.

Pero el volumen NO es un respaldo

Sobrevive al contenedor — no al host, ni al error humano, ni al ransomware



El host muere

La VM se corrompe, el disco falla, Play with Docker expira: el volumen vivía AHÍ. Se fue con el barco.



El error humano

docker volume rm equivocado, un prune agresivo, un script mal apuntado. El clásico.



Ransomware

Si cifran el host, cifran sus volúmenes. Los datos secuestrados junto con todo lo demás.

La regla 3-2-1 — el mínimo profesional:

3 copias de los datos · en 2 medios distintos · 1 fuera del sitio (offsite — la nube cuenta). Y la regla de oro encima: un respaldo que nunca has restaurado NO es un respaldo — es una esperanza.

Respaldar una base de datos containerizada

El export lógico: mysqldump a través de docker exec

```
docker exec db mysqldump -u root -p"$PASS" wordpress > respaldo.sql  
# el estado completo del blog, en un archivo de texto en el host
```

Sale del contenedor, vive en el host

El > escribe FUERA: respaldo.sql queda en tu carpeta, no en la capa efímera del contenedor.

Es texto: portable y versionable

Un .sql restaura en cualquier MySQL — otro contenedor, otra VM, otra nube. La portabilidad de siempre, ahora para los datos.

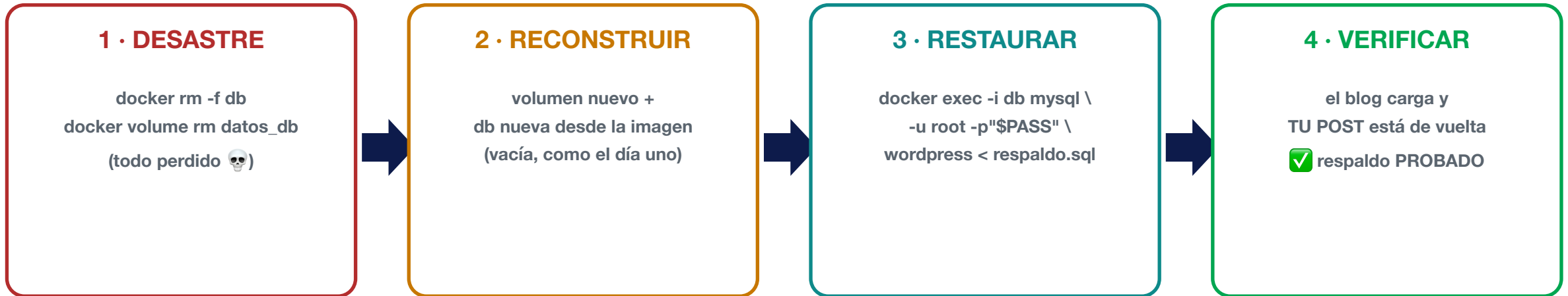
Para volúmenes que no son BD

El patrón tar: un contenedor efímero monta el volumen y lo empaca — `docker run --rm -v datos:/d -v $(pwd):/b alpine tar czf /b/datos.tgz /d`

Y ese archivo se lleva OFFSITE: a Azure Blob Storage, a otro equipo — la «1» de la regla 3-2-1.

Restaurar: el momento de la verdad

Hoy simulas el desastre TOTAL — y vuelves de él



RTO mental: ¿cuánto tardaste de «todo perdido» a «todo de vuelta»? Minutos.

Esa es la promesa de los contenedores + respaldos: la infraestructura es desechable y reconstruible; los datos, respaldados y probados.

Eje de seguridad · S4: los datos son el objetivo

Tres reglas para que el respaldo no sea el eslabón débil

1 · El respaldo también es dato sensible

Un respaldo.sql contiene TODO: usuarios, correos, contenido. Se protege como la base misma: cifrado en tránsito y en reposo, acceso mínimo. Jamás en un repo público.

2 · Offsite y fuera del alcance del atacante

El ransomware busca y cifra también los respaldos conectados. La copia offsite (Azure Blob, otra cuenta) con credenciales SEPARADAS es la que te salva. El nivel pro: almacenamiento inmutable (WORM).

3 · Restauración ensayada = plan real

La diferencia entre backup y recuperación es haberlo PRACTICADO. Hoy lo haces por primera vez; en una entidad, se ensaya con calendario.

En ciber-recuperación esto tiene nombre: resiliencia. El que tiene respaldo probado negocia distinto frente a un incidente.

Persistencia y respaldo en la nube

El mapeo híbrido de lo que hiciste hoy

docker volume

Azure: Managed Disk / Azure Files montado a la VM · AWS: EBS / EFS

respaldo.sql en el host

Azure Blob Storage (frío y barato) · AWS S3 — la copia offsite de la 3-2-1

restauración manual probada

Azure Backup / restore points automatizados con calendario y retención

bóveda anti-ransomware

Blob immutable storage (WORM) · S3 Object Lock — nadie borra ni cifra la copia

BD contenerizada con volumen

En producción real: BD gestionada (Azure Database for MySQL) — el respaldo viene incluido

La demo del docente hoy: el respaldo del blog subido a Azure Blob Storage — la copia offsite, de verdad.

Los comandos de hoy

La caja de herramientas de la persistencia

`docker volume create / ls / inspect / rm`

El ciclo de vida de un volumen — independiente de todo contenedor.

`docker run -v <volumen>:/ruta ...`

Monta el volumen en la ruta donde la app escribe sus datos.

`docker run -v $(pwd):/ruta ...`

Bind mount: una carpeta real del host dentro del contenedor.

`docker exec db mysqldump ... > respaldo.sql`

El export lógico de la BD hacia el host.

`docker exec -i db mysql ... < respaldo.sql`

La restauración (el -i es clave: alimenta el archivo por stdin).

`docker system df`

Cuánto ocupan imágenes, contenedores y volúmenes — FinOps local. 💰



Arranca el mini-proyecto integrador

Todo lo aprendido, en UNA aplicación tuya — cierre en la Sesión 5

La consigna: una aplicación multi-contenedor (app + base de datos), en red aislada, con datos persistentes en volumen, respaldo probado y su imagen publicada en Docker Hub.

FASE 1 · Esta semana

Arma tu app (el blog del lab u otra que elijas), con red + volumen + respaldo restaurado. Documenta los comandos en tu bitácora.

FASE 2 · Sesión 5 (cierre)

La migras: de Play with Docker a una VM de Azure (taller S3), publicando tu imagen. Demo final de 5 minutos: la app viva + la restauración en vivo.

En parejas o individual. La guía completa del mini-proyecto está en el repositorio del curso (mini-proyecto.md): consigna, fases, entregables y rúbrica.

El laboratorio de hoy

60 minutos · Play with Docker · del desastre a la resiliencia

Parte 1 · El experimento cruel + volúmenes (15 min)

Comprueba la pérdida: blog sin volumen, post, rm de la db... todo perdido. Luego el fix: volumen básico con alpine y la prueba de supervivencia.

Parte 2 · El blog inmortal (25 min)

El blog de la S3 con -v datos_db:/var/lib/mysql. Post → matar la db → recrearla montando el mismo volumen → el post sigue. La app es desechable; los datos no.

Parte 3 · Respaldo y restauración PROBADA (20 min)

mysqldump al host → desastre total (contenedor Y volumen) → reconstruir desde cero → restaurar el .sql → tu post vuelve. Cronometra tu RTO.

Entregable de la S4: evidencias del ciclo completo (pérdida → volumen → respaldo → restauración) + tu RTO cronometrado.

Las 3 ideas que te llevas hoy

1 • La capa de escritura muere con el contenedor: los datos importantes viven en **VOLÚMENES**.

2 • El volumen sobrevive al contenedor — el **RESPALDO** sobrevive al host. Necesitas ambos.

3 • Un respaldo sin restauración probada es una esperanza, no un plan.

 Próxima sesión — el cierre: portabilidad local → nube, introducción a Kubernetes y la demo final de tu mini-proyecto.